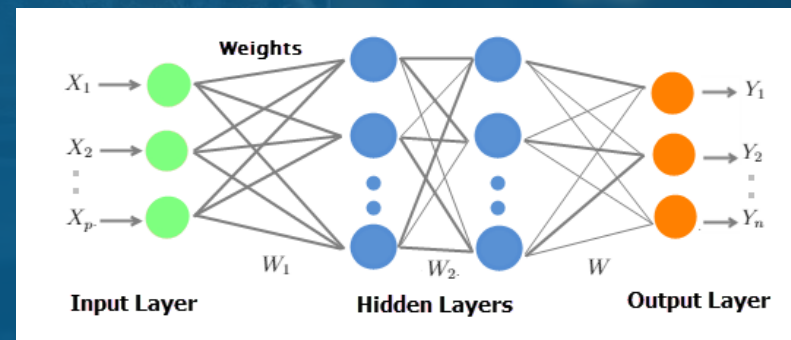


FQ01 TD 3

Approche par AI

Et si la qualité n'était plus seulement **contrôlée**... mais **prédite** ?



RAPPEL

On passe d'une :

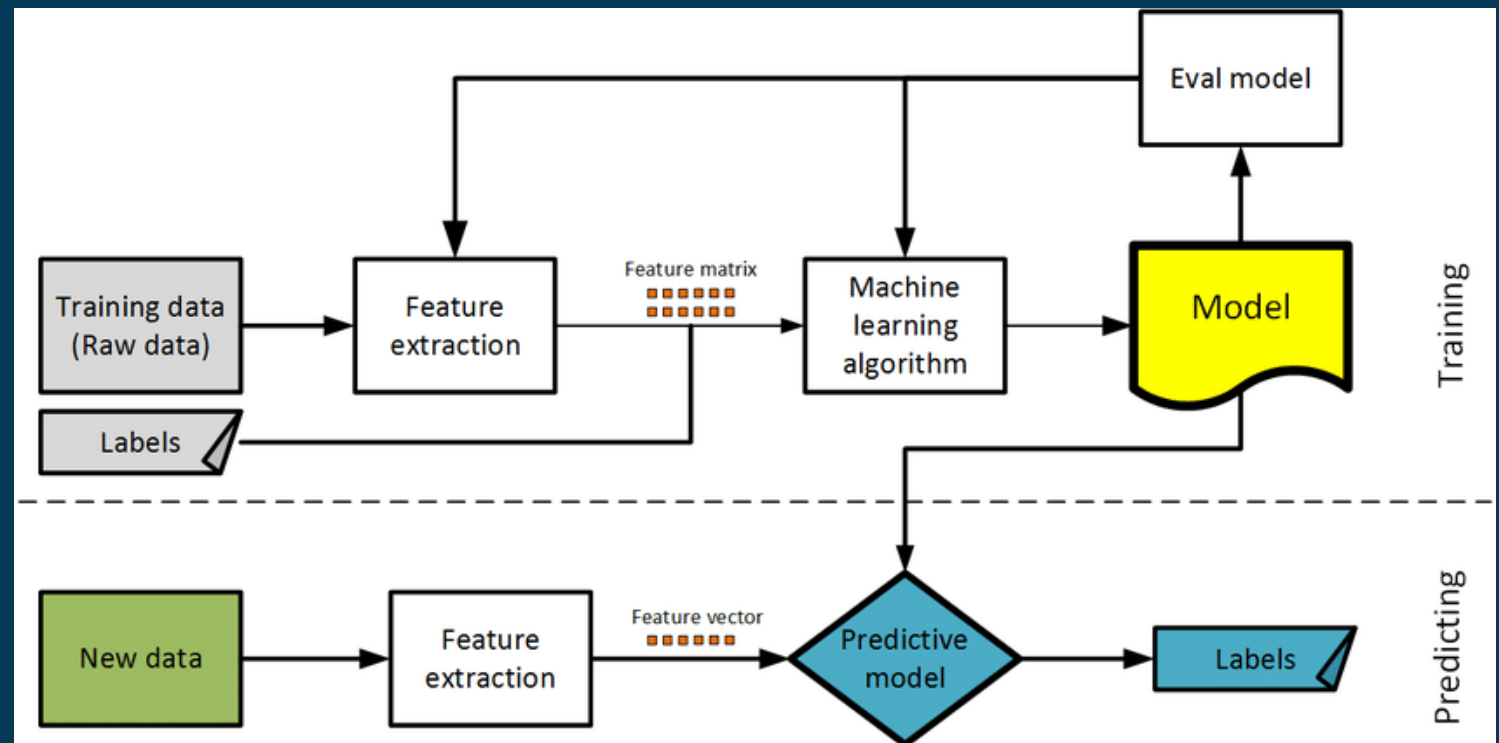
- **Qualité Réactive** à une **Qualité Prédictive**.
- **Approche traditionnelle** basée sur le **contrôle** à une **Approche prédictive et intelligente**.

Intelligence Artificielle : ensemble de Techniques permettant à une machine de :

1. Apprendre à partir des données
2. Prendre des décisions
3. Reconnaître des patterns
4. Prédire des non-conformités
5. Détecter les défauts
6. Optimiser les processus qualité

Objectifs de l'IA en qualité :

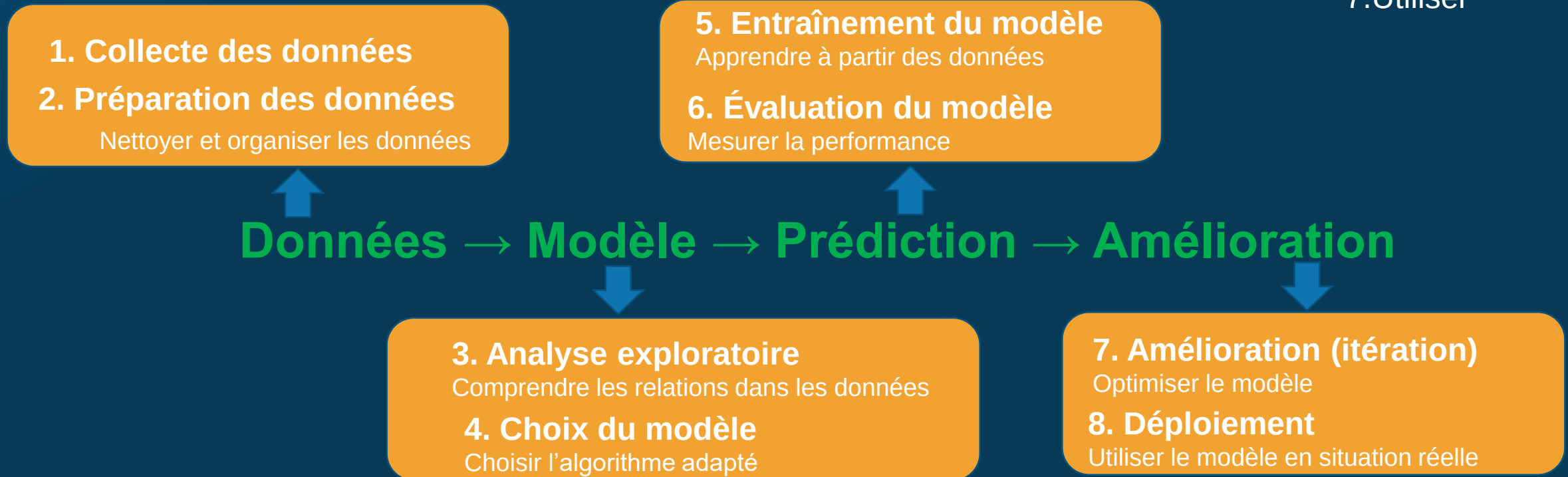
1. **Réduire** les défauts
2. **Améliorer** la prise de décision
3. **Automatiser** les contrôles
4. **Anticiper** les problèmes



RAPPEL

Le Machine Learning suit un processus logique :

1. Collecter
2. Nettoyer
3. Comprendre
4. Modéliser
5. Tester
6. Améliorer
7. Utiliser



Matrice de confusion

permet de comparer : **la réalité** et **la prédiction** du modèle

Exemple : prédiction de défaut

0 = pas de défaut

1 = défaut

	Prédit 0	Prédit 1
Réel 0	Vrai Négatif (VN)	Faux Positif (FP)
Réel 1	Faux Négatif (FN)	Vrai Positif (VP)

Vrai Positif (VP)

Le modèle prédit défaut
ET il y a vraiment un défaut
bonne prédiction

Faux Positif (FP)

Le modèle prédit défaut
MAIS il n'y a pas de défaut
fausse alarme

$$Accuracy = \frac{VP + VN}{Total}$$

$$Precision = \frac{VP}{VP + FP}$$

$$Recall = \frac{VP}{VP + FN}$$

Vrai Négatif (VN)

Le modèle prédit pas de défaut
ET effectivement il n'y a pas de défaut
bonne prédiction

Faux Négatif (FN)

Le modèle prédit pas de défaut
MAIS il y a un défaut
erreur grave

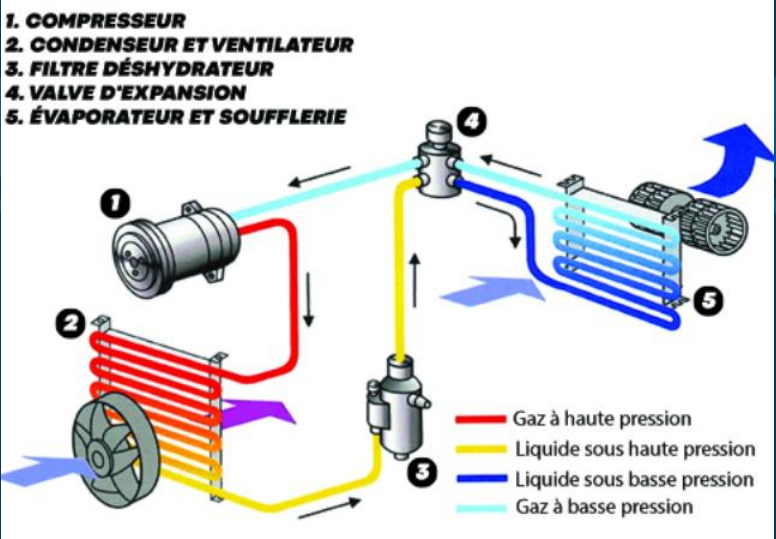
EXERCICE 1

Une entreprise industrielle automobile Toyota souhaite améliorer la qualité de sa production. Elle observe :

- 1. défauts sur certaines pièces
- 2. taux de non-qualité élevé
- 3. paramètres de production variables

Données des machines :

Variable	Description
temperature	Température de production (°C)
pressure	Pression (bar)
speed	Vitesse machine (rpm)
defect	Défaut (0 = conforme, 1 = défaut)



Question : Construire un modèle de Machine Learning (Régression logistique) pour prédire les défauts et améliorer la qualité

Livrables attendus

- code Python
 - graphiques
 - interprétation des résultats
- température > 200°C → risque ↑
 - pression > 5 bar → risque ↑
 - vitesse élevée > 1500 rmp → risque ↑
- faible risque → proche de 0
 - zone critique → transition rapide
 - fort risque → proche de 1

1. Import et création des données

pandas → manipulation des données (tableaux)
numpy → calcul numérique

sklearn.linear_model → régression logistique
matplotlib.pyplot

2. Génération des données (random) n = 1500 : X = Température, Pression et Vitesse

- moyenne = 200°C
- variation = ±10
- moyenne = 5 bar
- variation = ±1
- moyenne = 1500 rpm
- variation = ±200

y = 0 = pas de défaut ou 1 = défaut

3. Fonction **logistique** : transforme une valeur en probabilité entre 0 et 1

$$P(y = 1 | X) = \frac{1}{1 + e^{-(w_0 + w_1 \frac{x_1 - \mu_1}{\sigma_1} + w_2 \frac{x_2 - \mu_2}{\sigma_2} + \dots)}}$$

- μ_i = moyenne
- σ_i = écart-type (variation)

Modèle logistique (réaliste)

$$P = \frac{1}{1 + e^{-(2 + 0.05(T - 200) + 0.4(P - 5) + 0.002(V - 1500))}}$$

4. Séparation des données

- X = variables dépendantes
- y = variable indépendante cible

5. Prédiction du modèle

- défaut ou non
- 80% → apprentissage
- 20% → test

6. Visualisation

- X = température, pression, vitesse
- y = défaut

```

#Import des bibliothèques
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, mean_squared_error, r2_score

#Génération des données
n = 1500
temperature = np.random.normal(200, 10, n) # °C  moyenne=200 -  écart type ou variation=10
pressure = np.random.normal(5, 1, n)      # bar  moyenne=5    -  écart type = 1
speed = np.random.normal(1500, 200, n)    # rpm  moyenne=1500 - écart type = 200

#Modèle physique (probabilité de défaut)
z = - 2 + 0.05*(temperature - 200) + 0.4*(pressure - 5) + 0.002*(speed - 1500)
P = 1 / (1 + np.exp(-z))

#Génération de la variable défaut : 1 = nombre d'essais et P probabilité de succès
defect = np.random.binomial(1, P)

#Création du dataset
df = pd.DataFrame({
    "temperature": temperature,
    "pressure": pressure,
    "speed": speed,
    "defect": defect
})

#Préparation des données
X = df[["temperature", "pressure", "speed"]]
y = df["defect"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

```
#Modèle IA – Régression logistique et prediction
```

```
model = LogisticRegression()  
model.fit(X_train, y_train)  
y_pred = model.predict(X_test)
```

```
#Évaluation du modèle
```

```
mse = mean_squared_error(y_test, y_pred)  
r2 = r2_score(y_test, y_pred)  
AC = accuracy_score(y_test, y_pred)  
print("MSE :", mse)  
print("R2 :", r2)  
print("Accuracy :", AC )  
print(confusion_matrix(y_test, y_pred))
```

```
#Visualisation des donnees
```

```
plt.scatter(df["temperature"], df["defect"])  
plt.xlabel("Température (°C)")  
plt.ylabel("Défaut (0/1)")  
plt.title("Relation Température / Défaut")  
plt.show()
```

```
# Exemple : nouvelle situation
```

```
nouvelle_donnee = [[150, 3, 1300]] # temperature=150, pression = 3 et vitesse = 1300  
prediction = model.predict(nouvelle_donnee)  
print("Taux de défauts prédit :", prediction[0])
```

```
print("Matrice de confusion :")  
print(confusion_matrix(y_test, y_pred))
```


EXERCICE 2

Prédiction du temps d'attente à l'hôpital

Un hôpital souhaite améliorer la qualité de ses services, notamment en réduisant le temps d'attente des patients.

Des données ont été collectées sur :

1. le nombre de patients par jour
2. le nombre de médecins disponibles
3. le temps moyen de consultation



Construire un modèle de régression linéaire pour **prédire le temps d'attente des patients**

Livrables attendus :

1. Code Python
2. Graphiques
3. Interprétation des résultats

Modèle réaliste (avec bruit)

- a) patients $\uparrow \rightarrow$ attente $\uparrow$
- b) médecins $\uparrow \rightarrow$ attente $\downarrow$
- c) consultation $\uparrow \rightarrow$ attente $\uparrow$

Attendu avec random

• 1000 lignes

• variables réalistes :

- patients (50–200)
- médecins (5–20)
- consultation (~20 min)



$$\text{Attente} = 0.5 \cdot \text{Patients} - 2 \cdot \text{Medecins} + 1.2 \cdot \text{Consultation} + \varepsilon$$

le bruit ε : valeurs aléatoires suivant une **loi normale (gaussienne)**

n valeurs aléatoires centrées sur 0 avec une variabilité de 1

```
patients = np.random.randint(50, 200, n)
medecins = np.random.randint(5, 20, n)
consultation = np.random.normal(20, 5, n)
```

```
attente =
    0.5 * patients
    - 2 * medecins
    + 1.2 * consultation
    + np.random.normal(0, 10, n)
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
n = 1000
np.random.seed(42)
# Variables explicatives
patients = np.random.randint(50, 200, n)
medecins = np.random.randint(5, 20, n)
consultation = np.random.normal(20, 5, n)
```

```
# Modèle réaliste (avec bruit)
```

```
attente = (
    0.5 * patients
    - 2 * medecins
    + 1.2 * consultation
    + np.random.normal(0, 10, n)
)
```

```
# DataFrame
```

```
df = pd.DataFrame({
    "patients": patients,
    "medecins": medecins,
    "consultation": consultation,
    "attente": attente
})
```

```
#Préparation des données
```

```
X = df[["patients", "medecins", "consultation"]]
```

```
y = df["attente"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
#Modèle de régression linéaire
model = LinearRegression()
model.fit(X_train, y_train)

#Prédictions
y_pred = model.predict(X_test)

#Évaluation du modèle
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("MSE :", mse)
print("R2 :", r2)

#Analyse des coefficients
coefficients = pd.DataFrame({
    "Variable": ["patients", "medecins", "consultation"],
    "Coefficient": model.coef_
})

print(coefficients)
#Comparaison réel vs prédiction
plt.scatter(y_test, y_pred)
plt.xlabel("Valeurs réelles")
plt.ylabel("Valeurs prédites")
plt.title("Réel vs Prédit")
plt.show()
```

EXERCICE 3

Dans un procédé de soudage, les défauts dépendent de :

1. température de soudage T (°C)
2. vitesse de soudage V (mm/s)
3. humidité H (%)

Construire un modèle de Machine Learning pour prédire la probabilité de défaut



1 - Générer un dataset avec (random) $n = 1000$:

- a) Température (180 à 250°C)
- b) Vitesse (20 à 60)
- c) Humidité (30 à 80%)

```
T = np.random.uniform(180, 250, n)
V = np.random.uniform(20, 60, n)
H = np.random.uniform(30, 80, n)
```

< 180°C Mauvaise fusion entre passes
> 250°C Altération des propriétés mécaniques

< 20 mm/s → lent → risque défaut ↑
> 60 mm/s → rapide → risque défaut ↓

< 30% Risque accru d'humidité dans les consommables (électrodes, flux)
> 80% Augmentation de l'hydrogène diffusible → fissuration à froid

2 - Utiliser la formule de **probabilité** pour générer les défauts entre 0 et 1

- 3 - Séparer les données (train 80% /test 20%)
- 4 - Entraîner un modèle de régression logistique
- 5 - Faire des prédictions
- 6 - Calculer la précision
- 7 - Afficher la matrice de confusion
- 8 - Identifier les variables les plus importantes
- 9 - Proposer 2 actions qualité

$$P = \frac{1}{1 + e^{-(-3 + 0.04T - 0.02V + 0.05H)}}$$